

Shared Memory and Memory-Mapped Files

Introduction:

Shared memory and memory-mapped files are two methods used by the operating system for sharing data between processes. In a computer system, different processes normally have separate memory spaces. This is done for protection and security, because one process should not directly change the data of another process. However, there are situations where two or more processes need to exchange information or work on the same data. In such cases, the operating system provides different methods of inter-process communication.

Shared memory allows multiple processes to access a common region of memory. Instead of sending the complete data from one process to another, the processes can directly use the same memory area. Memory-mapped files work in a slightly different way. In this method, a file stored on the disk is mapped into the memory address space of a process. The process can then access the file using memory operations.

Both techniques can be useful when a program needs to handle a large amount of data or when multiple processes need to communicate efficiently.

IPC techniques:

There are in total 8 IPC techniques in which 2 are as follows:

1. Shared Memory:

Shared memory is an inter-process communication technique in which two or more processes share a common area of memory. The operating system creates or provides a memory region that can be accessed by the processes that are allowed to use it. Once the shared memory has been created, the processes can read data from it and write data to it.

The main advantage of shared memory is its speed. In many other IPC methods, data has to be copied from one process to another. With shared memory, both processes can access the same area directly. This can reduce the amount of data copying and make communication faster, especially when large amounts of information are being exchanged.

For example, one process may produce data and store it in the shared memory area. Another process can read the same data from that memory area and process it. The first process does not have to send the data again through a separate communication channel.

However, shared memory also creates a problem when more than one process tries to access or change the same data at the same time. If two processes modify the same memory location simultaneously, the final result may not be correct. This is known as a synchronization problem.

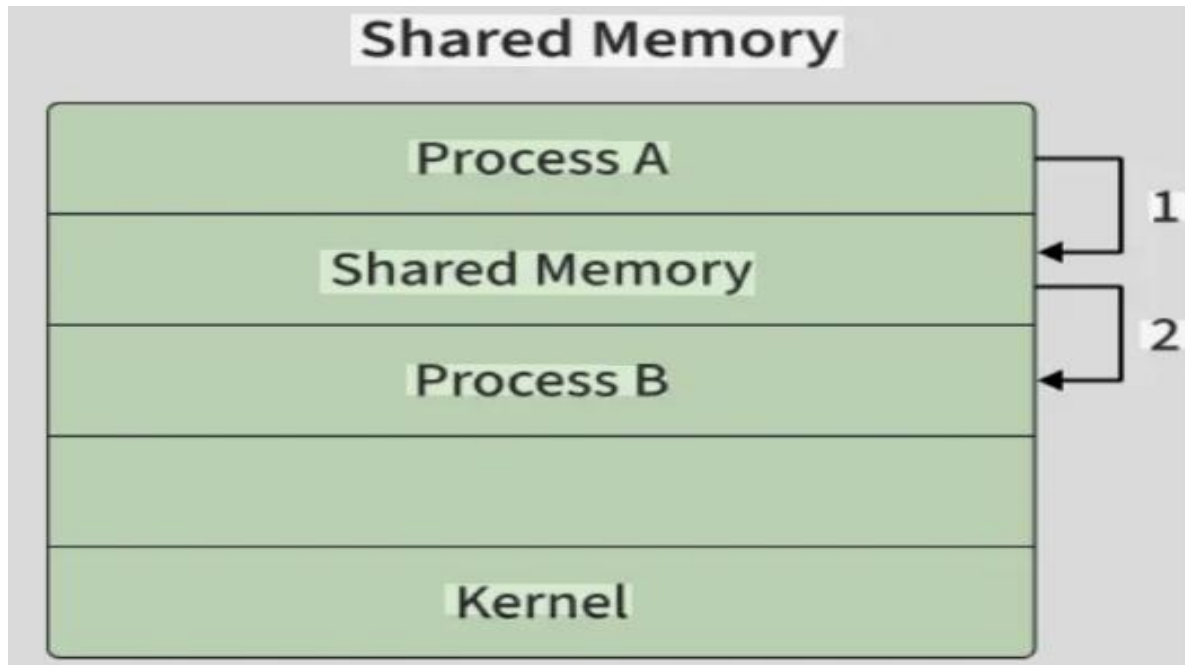


Figure 1 Shared Memory IPC diagram

To avoid synchronization problems, techniques such as semaphores, mutexes, or other locking mechanisms can be used. These techniques control which process is allowed to access the shared data at a particular time. For example, one process can lock the shared memory while it is updating the data, and another process can access it after the first process has finished.

Shared memory is therefore useful when high-speed communication between processes is required. It is commonly used in applications where processes need to exchange a large amount of information quickly.

Example: A producer process can generate a list of numbers and store them in a shared memory area. A consumer process can read these numbers from the same memory area and calculate their sum. Both processes are able to work with the same data without sending a separate copy of the numbers through another IPC mechanism.

Working of Shared Memory:

The working of shared memory can be understood in a few basic steps. First, a shared memory region is created by the operating system. A process can then attach

this memory region to its address space. Another process that has permission to access the same region can also attach it to its own address space.

After both processes have access to the shared memory, one process can write information into it and another process can read the information. The operating system manages the memory region and the permissions associated with it.

When the processes have finished using the shared memory, they can detach it. Depending on the operating system and the type of shared memory being used, the shared memory region can also be removed when it is no longer required.

Advantages of Shared Memory:

1. Shared memory provides fast communication between processes.
2. Large amounts of data can be exchanged without repeatedly copying the data between processes.
3. Multiple processes can access the same information.
4. It is useful for applications where processes need to communicate frequently.
5. It can reduce the overhead involved in transferring large amounts of data.

Limitations of Shared Memory:

1. Synchronization is required when multiple processes access the same data.
2. Incorrect use of shared memory can lead to data inconsistency.
3. It can be more difficult to program compared to simple communication methods.
4. Proper access permissions are required so that unauthorized processes cannot modify the shared data.
5. A programming error in one process can affect the data being used by another process.

2. Memory-Mapped Files:

A memory-mapped file is a file that is mapped into the memory address space of a process. Normally, when a program wants to use a file, it performs operations such as opening the file, reading its contents, modifying the data, and writing the data back to the file.

With memory mapping, the operating system creates a connection between the file and a region of the process's memory. The program can then access the contents of the file using memory addresses. The operating system handles the relationship between the memory region and the file stored on the disk.

Memory-mapped files are particularly useful when working with large files. Instead of manually reading the entire file into a separate buffer, a program can access the required portion through the mapped memory. The operating system can load the required portions of the file into physical memory when they are needed.

Memory-mapped files can also be used as a way for processes to share information. If multiple processes map the same file into their memory, they may be able to access common data through that file. This makes a memory-mapped file useful in some situations where processes need to work with the same information.

Example: Suppose a program has a large file containing student records. The program can map the file into its memory and access the records that it needs. If another process also maps the same file, it can access the same stored information. Changes made to the mapped data can eventually be reflected in the file according to the operating system's file-mapping behaviour.

Working of Memory-Mapped Files:

The process starts by opening the file that needs to be mapped. The operating system then creates a mapping between the file and a region of the process's virtual memory.

After the mapping is created, the program can access the contents of the file through the memory addresses associated with the mapping. When a particular part of the mapped file is needed, the operating system can bring the corresponding portion into physical memory.

If the program modifies the mapped memory, the changes can be written back to the file. The exact timing of these changes depends on the operating system and the way the mapping is managed.

When the file is no longer required, the process can remove the mapping and close the file.

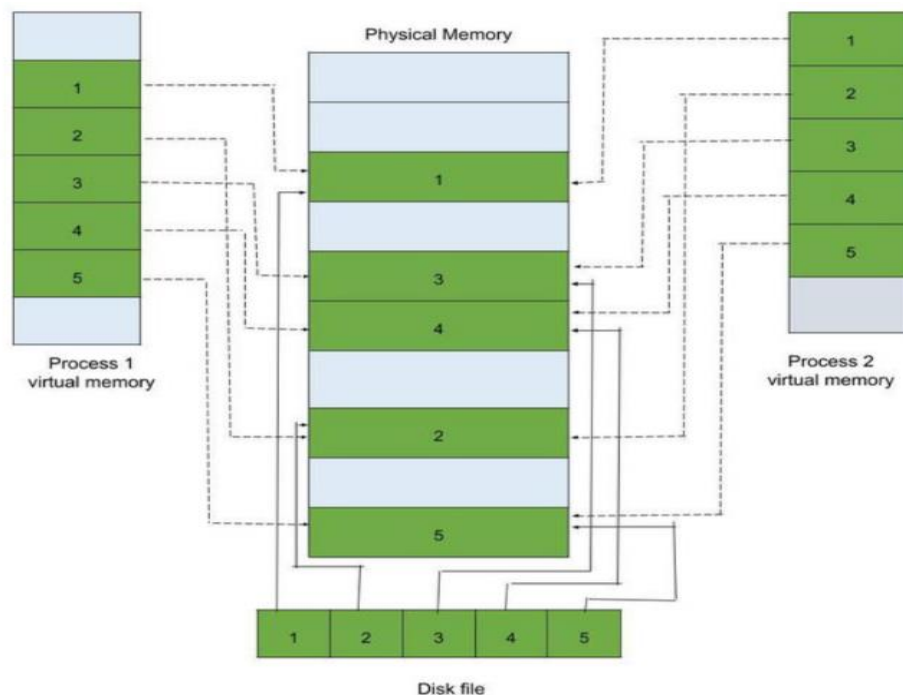


Figure 2. Working of Memory-Mapped Files

Uses of Memory-Mapped Files:

Memory-mapped files are useful in many situations where programs need to work with files efficiently. They can be used for handling large files, sharing data between processes, and accessing structured data.

They are also useful when a program needs to access different parts of a file repeatedly. Instead of repeatedly performing separate file read operations, the program can access the required data through the mapped memory region.

Memory-mapped files can also be useful for applications such as databases, data processing programs, operating system utilities, and programs that work with large datasets.

Advantages of Memory-Mapped Files:

1. They provide a convenient way to access the contents of a file through memory.
2. They can be useful when working with large files.
3. Multiple processes can potentially access the same mapped file.
4. They can reduce the need for repeated explicit file read and write operations.
5. Programs can access different portions of a file as needed.

Limitations of Memory-Mapped Files:

1. The programmer needs to understand how memory mapping works.
2. Synchronization may be required when multiple processes modify the same mapped data.
3. Changes to the mapped memory have to be managed properly to ensure that important data is stored correctly.
4. The size and structure of the file can affect how the mapping is used.
5. Improper handling of a mapped file can result in errors or incorrect data.

Difference Between Shared Memory and Memory-Mapped Files:

Shared memory and memory-mapped files are similar because both allow processes to access common data, but their main purpose and method of operation are different.

In shared memory, the operating system provides a region of memory that can be shared by multiple processes. The main purpose is usually fast communication between processes. The data exists in the shared memory region and can be directly accessed by the processes that are connected to it.

In memory-mapped files, a file is connected to a region of memory. The file provides persistent storage, while the mapped memory provides a convenient way for the program to access the file stored on disk.

Shared memory is generally used when processes need to exchange data quickly and frequently. Memory-mapped files are useful when processes need to work with file-based data or large files through memory access.

Both methods require proper synchronization when multiple processes can modify the same data at the same time. Without synchronization, one process may read incomplete or incorrect information while another process is changing it.

Difference Between Shared Memory and Memory-Mapped Files:

The main differences between shared memory and memory-mapped files can be understood based on how they store and share data.

Shared memory uses a common memory region that can be accessed by multiple processes. Its main purpose is communication between processes. It is useful when fast data exchange is required.

	Shared Memory	Distributed Memory
1	Distribution of data among the Cores or CPU is automatic.	Distribution of data is manually done by the programmer.
2	Communication and access to the memory is automatic and does not require programmer's intervention.	Requires programmers to explicitly define access to memory.
	Server CPUs or Cores have automatic access to the shared memory.	Each CPU or Core has its own Physical Memory and memory cache that is not available to the other cores unless defined by the programmer.
3	Requires extensive hardware support	Requires a simple network
4	Achieving high Performance is less complex as its less complicated to ensure message correctness.	Much more difficult to achieve high performance. Due to complexity in ensuring correct messaging.
5	Ensuring message correctness is less complex.	Ensure message correctness is complicated and improper messaging can case wrong distribution.

Figure 2 Difference between shared memory and memory-mapped files

Memory-mapped files use a file as the underlying source of data. The file is mapped into the virtual memory of a process, allowing the program to access the file through memory addresses. It is useful when programs need to work with large files or when file-based data needs to be accessed efficiently.

Another difference is persistence. Shared memory is mainly used as a temporary communication area between processes, while a memory-mapped file is connected to a file that can remain stored on disk.

In both cases, access must be managed carefully when multiple processes are working with the same data. Synchronization mechanisms may be needed to prevent conflicts and maintain correct information.

Conclusion:

Shared memory and memory-mapped files are important techniques for handling data between processes. Shared memory allows processes to directly access a common memory region and is useful for fast communication. Memory-mapped files allow a file to be accessed through memory and are useful for working with large files and sharing file-based data.

Both techniques can improve the efficiency of programs when used correctly. At the same time, they require proper management of memory, permissions, and synchronization. Understanding these techniques helps in designing programs that need to exchange data between different processes efficiently.